Run samples of the ServiceQueue simulation

Collect statistics and plot histograms along the way.

```
PictureFolder = "Pictures";
mkdir(PictureFolder);
```

```
Warning: Directory already exists.
```

Set up

We'll measure time in minutes (need to change code from hours to minutes *****)

Arrival rate: 1 every 75 seconds = 1 every 1.25 minutes = 1 per 1.25 minutes = 1/1.25 = 0.8

These peak periods last about four hours. So we need to multiply by 4 hours = 4(60) = 240 minutes

lambda (4 hours)(60 min) = 0.8(240) = 192 total number of customers in 4 hours or 240 minutes

```
lambda = 0.8;
```

Departure (service) rate: 6.5 minutes for each customer to be served

mu = 1/6.5 = 0.1538

```
mu = 0.1538;
```

Calculating the Number of Servers to Iterate Thru

k = s: number of servers

We are told that there's room for up to eight registers in the checkout area.

Since we can have up to 8 servers, we know that k can go thru 1 to 8.

However, we can further narrow down our code using some theory.

service rate: k*mu

lambda: cutomer arrival rate

We want the service rate to be greater than the customer arrival rate to be successful.

k *mu <= lambda means that the system is not stable -- means customers are arriving faster than departing

k *mu > lambda means that the system is stable -- means customers are departing faster than arriving

5*mu = 5(0.1538) = 0.769 < .8 = lambda -- not a stable system

6* mu = 6(0.1538) = 0.9228 > .8 = lambda -- system is stable

This is why we will iterate our loop for k = 6:8.

rho = (lambda)/(k*mu) < 1

is our desired ratio to keep the system stable.

We will need

k > (lambda)/(mu)

many servers for a stable system.

k > (0.8)/(0.1538) \approx 5.2

The minimum number of servers that we will need is 6 for our system to be stable.

Run 100 samples of the queue.

The number of samples represents the number of shifts that we simulate.

```
NumSamples = 100;
```

Each sample is run up to a maximum time.

Run each sample of the queue for a simulated time of 4 hours to match the given length of the peak period.

These peak periods last about four hours. So we need to multiply by 4 hours = 4(60) = 240 minutes

```
MaxTime = 240;
```

Make a log entry every so often

Change the LogInterval to 1 every 5 minutes.

```
LogInterval = 5/60;
```

Numbers from theory for M/M/1 queue

The Formulas Used for P_0 and P_n

$\f{1}{P_0} = \sum \limits ^{k-1}_{n=0} \f{(\f{\lambda}{\mu})^n}{n!} + \f{1}{k!}(\f{\lambda}{\mu})^k\cdot \f{1}{1-\f{\lambda}{k\mu}}$

$P_{n+1 } = P_n   (\f{\lambda_n}{mu_{n+1}}) $

You may find the formulas for L_q, W_q, L, and W in the code and straight forward.

Update the calculation of the ►► ’s so that it works for any number ► of registers (serving stations).

Theoretical Calcultaions for Calculations of P_n, L_q, W_q, L, W

n: total number of customers in the system

The n values are calucated based upon the k values.

n < k : There are empty registers. Each new customer goes straight to a cashier.

n \geq k : All registers are full. Any new customer must wait.

k = 6 , 7, 8

n = 0, 1, 2, 3, 4, 5, 6

set up keeps us in the n < k state as much as possible with the only concern when 6 = n \geq k = 6

This is why n = 1:6.

```
for k = 6:8
    P = zeros(1, k-1);    % initiaizes the row vector the first time
    % clears row vector so I can use this again for the next graph
    % thereafter


fprintf("*********************************************************************
***");

fprintf("**
 **");
     fprintf("**          Theoretical Calculations for P_n, L_q, W_q, L,
W         **");
     fprintf("**                        " + "number of servers k = " + k+
"                      **");

fprintf("**
 **");

fprintf("*********************************************************************
***");


    disp(" ");
```

```
    %fprintf("The current number of servers in the theoretical calculations
is\nk = %d", k);


j = 0:(k-1);
terms = ((lambda / mu).^j) ./ factorial(j);
first = sum(terms);
second = (1 / factorial(k)) * (lambda / mu).^k * (1 / (1 - lambda / (k *
mu)));
P0 = 1/(first + second);
fprintf("P(0) = %f\n", P0);
P(1) = P0;


%%%  FIND A WAY TO SAVE ALL THE P(Wq??? > 5) \leq .1   info and just print
%%%  that here ***************

nMax = k-1;
    fprintf("The probability that some cashiers are busy but there is no
line:");
for n = 1:nMax


        if  n < k             % 1 <= n <= k

            P(n+1) = (1./ factorial(n))*((lambda / mu).^n)*(P0);
            fprintf("P(%d) = %f\n", n, P(n+1));

        else     %  k <= n

        fprintf("The probability that all cashiers are busy \n" + ...
            "and customers are either being served or waiting in line:");
        P(n+1) = (1./ factorial(k) * k^(n-k))*((lambda / mu).^n)*(P0);
        fprintf("P(%d) = %f\n", n, P(n+1));

        end

end

      top = ((lambda / mu).^k)*(lambda / (k*mu));
      bottom = factorial(k) *(1- (lambda / (k*mu))).^2;
      Lq = (top./ bottom)*(P0);                           % expected number
of customers waiting
      fprintf("The number of customers waiting, L_q(%d): %f\n", n,   Lq);
      L = Lq + (lambda./mu);
      fprintf("The expected number of customers in the system, L(%d): %f\n",
n,  L);
      Wq = (Lq./ lambda);
      fprintf("The expected time customers wait before beginning service,
W_q(%d): %f\n", n,  Wq);
```

```
      W = (L./ lambda);
      fprintf("The  expected time customers spend in the system, including
waiting time and service time, W(%d): %f\n", n,  W);




    disp(" ");  % this is also needed for aesthetic reasons
    disp(" ");
```

```
**********************************************************************
**                                                                  **
**          Theoretical Calculations for P_n, L_q, W_q, L, W        **
**                      number of servers k = 6                     **
**                                                                  **
**********************************************************************

P(0) = 0.003204
The probability that some cashiers are busy but there is no line:
P(1) = 0.016664
P(2) = 0.043340
P(3) = 0.075146
P(4) = 0.097719
P(5) = 0.101658
The number of customers waiting, L_q(5): 4.314453
The expected number of customers in the system, L(5): 9.516013
The expected time customers wait before beginning service, W_q(5): 5.393066
The  expected time customers spend in the system, including waiting time and service time, W(5): 11.895016


**********************************************************************
**                                                                  **
**          Theoretical Calculations for P_n, L_q, W_q, L, W        **
**                      number of servers k = 7                     **
**                                                                  **
**********************************************************************

P(0) = 0.004706
The probability that some cashiers are busy but there is no line:
P(1) = 0.024479
P(2) = 0.063665
P(3) = 0.110386
P(4) = 0.143544
P(5) = 0.149331
P(6) = 0.129459
The number of customers waiting, L_q(6): 1.082949
The expected number of customers in the system, L(6): 6.284510
The expected time customers wait before beginning service, W_q(6): 1.353686
The  expected time customers spend in the system, including waiting time and service time, W(6): 7.855637


**********************************************************************
**                                                                  **
**          Theoretical Calculations for P_n, L_q, W_q, L, W        **
**                      number of servers k = 8                     **
**                                                                  **
**********************************************************************
```

```
P(0) = 0.005226
The probability that some cashiers are busy but there is no line:
P(1) = 0.027182
P(2) = 0.070694
P(3) = 0.122572
P(4) = 0.159392
P(5) = 0.165817
P(6) = 0.143751
P(7) = 0.106819
The number of customers waiting, L_q(7): 0.369048
The expected number of customers in the system, L(7): 5.570609
The expected time customers wait before beginning service, W_q(7): 0.461310
The  expected time customers spend in the system, including waiting time and service time, W(7): 6.963261
```

Running simulation samples

This is the most time consuming calculation in the script, so let's put it in its own section. That way, we can run it once, and more easily run the faster calculations multiple times as we add features to this script.

Reset the random number generator. This causes MATLAB to use the same sequence of pseudo-random numbers each time you run the script, which means the results come out exactly the same. This is a good idea for testing purposes. Under other circumstances, you probably want the random numbers to be truly unpredictable and you wouldn't do this.

```
rng("default");
```

We'll store our queue simulation objects in this list.

```
QSamples = cell([NumSamples, 1]);
```

The statistics come out weird if the log interval is too short, because the log entries are not independent enough. So the log interval should be long enough for several arrival and departure events happen.

```
s = k;



fprintf("*********************************************************************
***");

fprintf("**
 **");
     fprintf("**           Simulation Calculations for P_n, L_q, W_q, L,
W          **");
     fprintf("**                        " + "number of servers k = " + k+
"                      **");

fprintf("**
 **");
```

```
fprintf("*********************************************************************
***");

    disp("");
for SampleNum = 1:NumSamples
    fprintf("Working on sample %d\n", SampleNum);
    q = ServiceQueue( ...
        ArrivalRate=lambda, ...
        DepartureRate=mu, ...
        NumServers=s, ...
        LogInterval=LogInterval);
    q.schedule_event(Arrival(random(q.InterArrivalDist), Customer(1)));
    run_until(q, MaxTime);
    QSamples{SampleNum} = q;
end
```

```
*********************************************************************
**                                                                 **
**          Simulation Calculations for P_n, L_q, W_q, L, W        **
**                      number of servers k = 6                    **
**                                                                 **
*********************************************************************
Working on sample 1
Working on sample 2
Working on sample 3
Working on sample 4
Working on sample 5
Working on sample 6
Working on sample 7
Working on sample 8
Working on sample 9
Working on sample 10
Working on sample 11
Working on sample 12
Working on sample 13
Working on sample 14
Working on sample 15
Working on sample 16
Working on sample 17
Working on sample 18
Working on sample 19
Working on sample 20
Working on sample 21
Working on sample 22
Working on sample 23
Working on sample 24
Working on sample 25
Working on sample 26
Working on sample 27
Working on sample 28
Working on sample 29
Working on sample 30
Working on sample 31
Working on sample 32
Working on sample 33
Working on sample 34
```

```
Working on sample 35
Working on sample 36
Working on sample 37
Working on sample 38
Working on sample 39
Working on sample 40
Working on sample 41
Working on sample 42
Working on sample 43
Working on sample 44
Working on sample 45
Working on sample 46
Working on sample 47
Working on sample 48
Working on sample 49
Working on sample 50
Working on sample 51
Working on sample 52
Working on sample 53
Working on sample 54
Working on sample 55
Working on sample 56
Working on sample 57
Working on sample 58
Working on sample 59
Working on sample 60
Working on sample 61
Working on sample 62
Working on sample 63
Working on sample 64
Working on sample 65
Working on sample 66
Working on sample 67
Working on sample 68
Working on sample 69
Working on sample 70
Working on sample 71
Working on sample 72
Working on sample 73
Working on sample 74
Working on sample 75
Working on sample 76
Working on sample 77
Working on sample 78
Working on sample 79
Working on sample 80
Working on sample 81
Working on sample 82
Working on sample 83
Working on sample 84
Working on sample 85
Working on sample 86
Working on sample 87
Working on sample 88
Working on sample 89
Working on sample 90
Working on sample 91
Working on sample 92
Working on sample 93
Working on sample 94
Working on sample 95
Working on sample 96
Working on sample 97
Working on sample 98
```

```
Working on sample 99
Working on sample 100
*********************************************************************
**                                                                 **
**          Simulation Calculations for P_n, L_q, W_q, L, W        **
**                      number of servers k = 7                    **
**                                                                 **
*********************************************************************
Working on sample 1
Working on sample 2
Working on sample 3
Working on sample 4
Working on sample 5
Working on sample 6
Working on sample 7
Working on sample 8
Working on sample 9
Working on sample 10
Working on sample 11
Working on sample 12
Working on sample 13
Working on sample 14
Working on sample 15
Working on sample 16
Working on sample 17
Working on sample 18
Working on sample 19
Working on sample 20
Working on sample 21
Working on sample 22
Working on sample 23
Working on sample 24
Working on sample 25
Working on sample 26
Working on sample 27
Working on sample 28
Working on sample 29
Working on sample 30
Working on sample 31
Working on sample 32
Working on sample 33
Working on sample 34
Working on sample 35
Working on sample 36
Working on sample 37
Working on sample 38
Working on sample 39
Working on sample 40
Working on sample 41
Working on sample 42
Working on sample 43
Working on sample 44
Working on sample 45
Working on sample 46
Working on sample 47
Working on sample 48
Working on sample 49
Working on sample 50
Working on sample 51
Working on sample 52
Working on sample 53
Working on sample 54
Working on sample 55
Working on sample 56
```

```
Working on sample 57
Working on sample 58
Working on sample 59
Working on sample 60
Working on sample 61
Working on sample 62
Working on sample 63
Working on sample 64
Working on sample 65
Working on sample 66
Working on sample 67
Working on sample 68
Working on sample 69
Working on sample 70
Working on sample 71
Working on sample 72
Working on sample 73
Working on sample 74
Working on sample 75
Working on sample 76
Working on sample 77
Working on sample 78
Working on sample 79
Working on sample 80
Working on sample 81
Working on sample 82
Working on sample 83
Working on sample 84
Working on sample 85
Working on sample 86
Working on sample 87
Working on sample 88
Working on sample 89
Working on sample 90
Working on sample 91
Working on sample 92
Working on sample 93
Working on sample 94
Working on sample 95
Working on sample 96
Working on sample 97
Working on sample 98
Working on sample 99
Working on sample 100
**********************************************************************
**                                                                  **
**          Simulation Calculations for P_n, L_q, W_q, L, W         **
**                       number of servers k = 8                    **
**                                                                  **
**********************************************************************
Working on sample 1
Working on sample 2
Working on sample 3
Working on sample 4
Working on sample 5
Working on sample 6
Working on sample 7
Working on sample 8
Working on sample 9
Working on sample 10
Working on sample 11
Working on sample 12
Working on sample 13
Working on sample 14
```

```
Working on sample 15
Working on sample 16
Working on sample 17
Working on sample 18
Working on sample 19
Working on sample 20
Working on sample 21
Working on sample 22
Working on sample 23
Working on sample 24
Working on sample 25
Working on sample 26
Working on sample 27
Working on sample 28
Working on sample 29
Working on sample 30
Working on sample 31
Working on sample 32
Working on sample 33
Working on sample 34
Working on sample 35
Working on sample 36
Working on sample 37
Working on sample 38
Working on sample 39
Working on sample 40
Working on sample 41
Working on sample 42
Working on sample 43
Working on sample 44
Working on sample 45
Working on sample 46
Working on sample 47
Working on sample 48
Working on sample 49
Working on sample 50
Working on sample 51
Working on sample 52
Working on sample 53
Working on sample 54
Working on sample 55
Working on sample 56
Working on sample 57
Working on sample 58
Working on sample 59
Working on sample 60
Working on sample 61
Working on sample 62
Working on sample 63
Working on sample 64
Working on sample 65
Working on sample 66
Working on sample 67
Working on sample 68
Working on sample 69
Working on sample 70
Working on sample 71
Working on sample 72
Working on sample 73
Working on sample 74
Working on sample 75
Working on sample 76
Working on sample 77
Working on sample 78
```

```
Working on sample 79
Working on sample 80
Working on sample 81
Working on sample 82
Working on sample 83
Working on sample 84
Working on sample 85
Working on sample 86
Working on sample 87
Working on sample 88
Working on sample 89
Working on sample 90
Working on sample 91
Working on sample 92
Working on sample 93
Working on sample 94
Working on sample 95
Working on sample 96
Working on sample 97
Working on sample 98
Working on sample 99
Working on sample 100
```

Collect measurements of how many customers are in the system

Count how many customers are in the system at each log entry for each sample run. There are two ways to do this. You only have to do one of them.

Option one: Use a for loop.

```
NumInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % Pull out samples of the number of customers in the queue system. Each
    % sample run of the queue results in a column of samples of customer
    % counts, because tables like q.Log allow easy extraction of whole
    % columns like this.
    NumInSystemSamples{SampleNum} = q.Log.NumWaiting + q.Log.NumInService;
end
```

System Calculation for L_q

(Number of Customers Waiting in the System)

```
NumWaitingInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    NumWaitingInSystemSamples{SampleNum} = q.Log.NumWaiting;
end
```

```
NumWaitingInSystemSamples = cellfun(@(q) q.Log.NumWaiting, ...
    QSamples, UniformOutput= false);

NumWaiting = vertcat(NumWaitingInSystemSamples{:});
meanNumWaiting = mean(NumWaiting);
%fprintf("Mean number waiting (L_q): %f\n", meanNumWaiting);
```

System W_q and L Calculation

Ricky, the customer relations specialist, says that during these peak periods, they'd like to have no more than 10% of customers wait more than 5 minutes before reaching a cashier.

customer waiting in the queue (W_q)

P(W_q > 5) \le 0.10

%% $$P(W_q > t) = C(s, \rho) e^{-s\mu(1-\rho)t}$$

```
WaitingTimeInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % The next command has many parts.
    %
    % q.Served is a row vector of all customers served in this particular
    % sample.
    % The ' on q.Served' transposes it to a column.
    %
    % The @(c) ... expression below says given a customer c, compute its
    % departure time minus its arrival time, which is how long c spent in
    % the system.
    %
    % cellfun(@(c) ..., q.Served') means to compute the time each customer
    % in q.Served spent in the system, and build a column vector of the
    % results.
    %
    % The column vector is stored in TimeInSystemSamples{SampleNum}.
    WaitingTimeInSystemSamples{SampleNum} = ...
        cellfun(@(c) c.BeginServiceTime - c.ArrivalTime , q.Served');
end
WaitingTimeInSystemSamples = cellfun( ...
    @(q) cellfun(@(c) c.BeginServiceTime - c.ArrivalTime , q.Served'), ...
    QSamples, ...
    UniformOutput=false);
ExpectedWaitingTime = vertcat(WaitingTimeInSystemSamples{:});
```

```
meanWaitTime = mean(ExpectedWaitingTime);
%fprintf("Mean waiting time (W_q): %f\n", meanWaitTime);
```

Option two: Map a function over the cell array of ServiceQueue objects.

The `@(q) ...` expression is shorthand for a function that takes a `ServiceQueue` as input, names it `q`, and computes the sum of two columns from its log. The `cellfun` function applies that function to each item in `QSamples`. The option `UniformOutput=false` tells `cellfun` to produce a cell array rather than a numerical array.

```
NumInSystemSamples = cellfun( ...
    @(q) q.Log.NumWaiting + q.Log.NumInService, ...
    QSamples, ...
    UniformOutput=false);
```

Join numbers from all sample runs.

`vertcat` is short for "vertical concatenate", meaning it joins a bunch of arrays vertically, which in this case results in one tall column.

```
NumInSystem = vertcat(NumInSystemSamples{:});
```

MATLAB-ism: When you pull multiple items from a cell array, the result is a "comma-separated list" rather than some kind of array. Thus, the above means

`NumInSystem = vertcat(NumInSystemSamples{1}, NumInSystemSamples{2}, ...)`

which concatenates all the columns of numbers in NumInSystemSamples into one long column.

This is roughly equivalent to "splatting" in Python, which looks like `f(*args)`.

Pictures and stats for number of customers in system

Print out mean number of customers in the system.

```
meanNumInSystem = mean(NumInSystem);
fprintf("Mean number in system: %f\n", meanNumInSystem);
```

Make a figure with one set of axes.

```
fig = figure();
t = tiledlayout(fig,1,1);
ax = nexttile(t);
```

MATLAB-ism: Once you've created a picture, you can use `hold` to cause further plotting functions to work with the same picture rather than create a new one.

```
hold(ax, "on");
```

Start with a histogram. The result is an empirical PDF, that is, the area of the bar at horizontal index n is proportional to the fraction of samples for which there were n customers in the system. The data for this histogram is counts of customers, which must all be whole numbers. The option `BinMethod="integers"` means to use bins $(-0.5, 0.5), (0.5, 1.5), \ldots$ so that the height of the first bar is proportional to the count of 0s in the data, the height of the second bar is proportional to the count of 1s, etc. MATLAB can choose bins automatically, but since we know the data consists of whole numbers, it makes sense to specify this option so we get consistent results.

```
h = histogram(ax, NumInSystem, Normalization="probability",
BinMethod="integers");
```

Plot $(0, P_0), (1, P_1), \ldots$. If all goes well, these dots should land close to the tops of the bars of the histogram.

```
% Since I only want certain values from the matrix P, I am collecting them
% so that the graph will work properly
%if s == 6
   % PP = P(1:7);

%elseif s == 7
    %PP = P(8:14);

%elseif s == 8
    %PP = P(15:21);

%end
    fprintf("These are the y axis P values:");
    disp(P);
    %fprintf("This is PP:");
    %disp(PP);

plot(ax, 0:nMax, P, 'o', MarkerEdgeColor='k', MarkerFaceColor='r');
```

Add titles and labels and such.

```
title(ax, "Number of customers in the system when s = " + s);
xlabel(ax, "Count");
ylabel(ax, "Probability");
legend(ax, "simulation", "theory");
```

Set ranges on the axes. MATLAB's plotting functions do this automatically, but when you need to compare two sets of data, it's a good idea to use the same ranges on the two pictures. To start, you can let MATLAB choose the ranges automatically, and just know that it might choose very different ranges for different sets of data. Once you're certain the picture content is correct, choose an x range and a y range that gives good results for all sets of data. The final choice of ranges is a matter of some trial and error. You generally have to do these commands *after* calling `plot` and `histogram`.

This sets the vertical axis to go from 0 to 0.3.

```
ylim(ax, [0, 0.18]);   % ******** I changed this from .3 to .18
```

This sets the horizontal axis to go from -1 to 21. The histogram will use bins $(-0.5, 0.5), (0.5, 1.5), \ldots$ so this leaves some visual breathing room on the left.

```
xlim(ax, [-1, 15 ]);      % ******** I changed this from 21 to 15
```

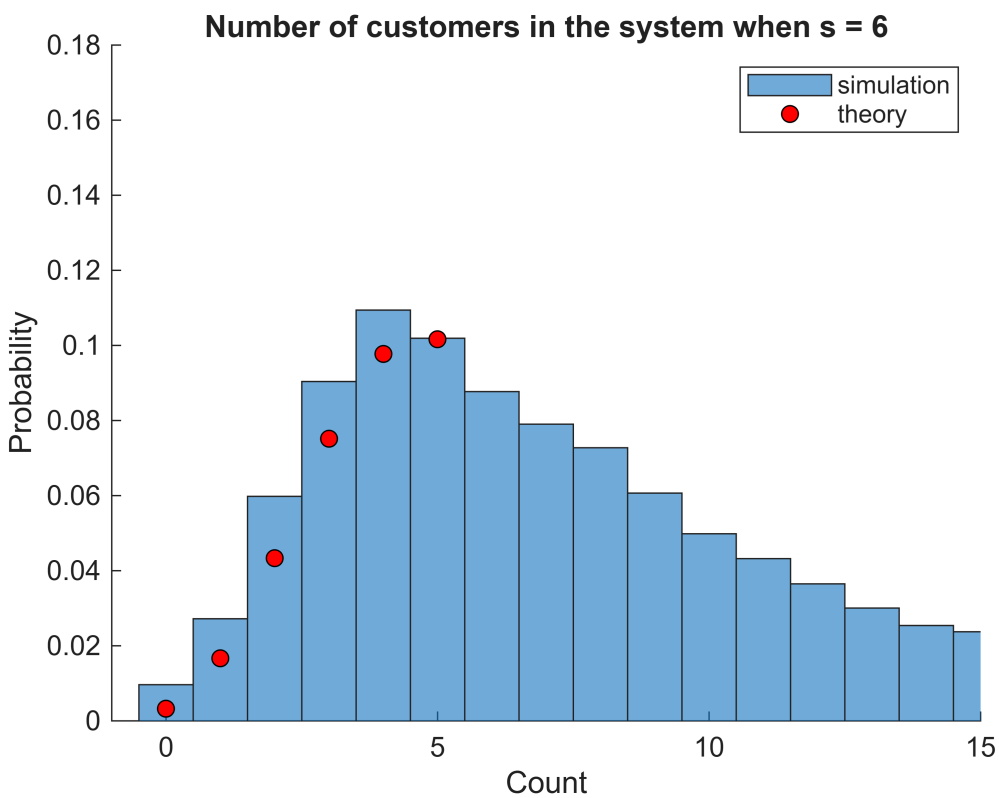
MATLAB-ism: You have to wait a couple of seconds for those settings to take effect or `exportgraphics` will screw up the margins.

```
pause(2);
```

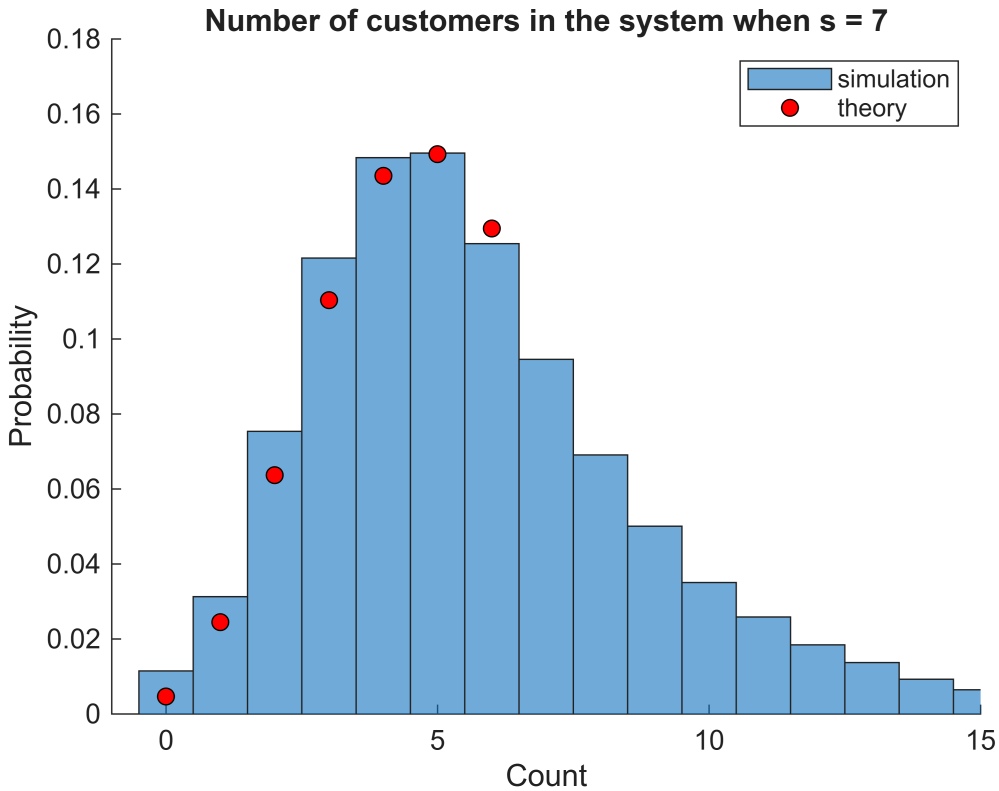
Save the picture.

```
exportgraphics(fig, PictureFolder + filesep + "Number in system
histogram.pdf");
exportgraphics(fig, PictureFolder + filesep + "Number in system
histogram.svg");
```

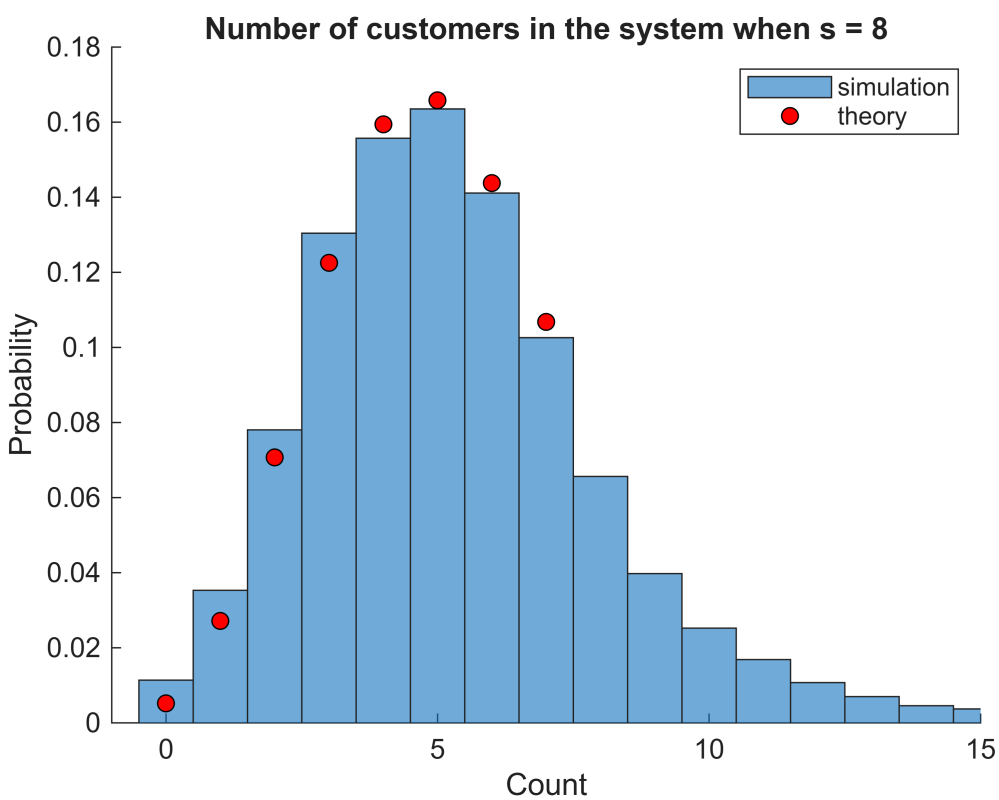
```
Mean number in system: 7.873257
These are the y axis P values:
    0.0032    0.0167    0.0433    0.0751    0.0977    0.1017
```




```
Mean number in system: 5.825295
These are the y axis P values:
    0.0047    0.0245    0.0637    0.1104    0.1435    0.1493    0.1295
```




```
Mean number in system: 5.401927
These are the y axis P values:
    0.0052    0.0272    0.0707    0.1226    0.1594    0.1658    0.1438    0.1068
```



Collect measurements of how long customers spend in the system

This is a rather different calculation because instead of looking at log entries for each sample `ServiceQueue`, we'll look at the list of served customers in each sample `ServiceQueue`.

Option one: Use a for loop.

```
TimeInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % The next command has many parts.
    %
    % q.Served is a row vector of all customers served in this particular
    % sample.
    % The ' on q.Served' transposes it to a column.
    %
    % The @(c) ... expression below says given a customer c, compute its
    % departure time minus its arrival time, which is how long c spent in
    % the system.
    %
    % cellfun(@(c) ..., q.Served') means to compute the time each customer
    % in q.Served spent in the system, and build a column vector of the
    % results.
    %
    % The column vector is stored in TimeInSystemSamples{SampleNum}.
    TimeInSystemSamples{SampleNum} = ...
        cellfun(@(c) c.DepartureTime - c.ArrivalTime, q.Served');
end
```

Option two: Use `cellfun` twice.

The outer call to `cellfun` means do something to each `ServiceQueue` object in `QSamples`. The "something" it does is to look at each customer in the `ServiceQueue` object's list `q.Served` and compute the time it spent in the system.

```
TimeInSystemSamples = cellfun( ...
    @(q) cellfun(@(c) c.DepartureTime - c.ArrivalTime, q.Served'), ...
    QSamples, ...
    UniformOutput=false);
```

Join them all into one big column.

```
TimeInSystem = vertcat(TimeInSystemSamples{:});
%fprintf("Total time customer spends in system (W): %f\n", TimeInSystem);
```

Pictures and stats for time customers spend in the system

Print out mean time spent in the system.

```
meanTimeInSystem = mean(TimeInSystem);
fprintf("Mean time in system: %f\n", meanTimeInSystem);
```

Make a figure with one set of axes.

```
fig = figure();
t = tiledlayout(fig,1,1);
ax = nexttile(t);
```

This time, the data is a list of real numbers, not integers. The option `BinWidth=...` means to use bins of a particular width, and choose the left-most and right-most edges automatically. Instead, you could specify the left-most and right-most edges explicitly. For instance, using `BinEdges=0:0.5:60` means to use bins $(0, 0.5), (0.5, 1.0), \ldots$

```
h = histogram(ax, TimeInSystem, Normalization="probability", BinWidth=5/60);
```

Add titles and labels and such.

```
title(ax, "Time in the system when s = "+ s);
xlabel(ax, "Time");
ylabel(ax, "Probability");
```

Set ranges on the axes.

```
ylim(ax, [0, 0.02]);            % *********changed this from .2 to .02
xlim(ax, [0, 3.0]);             % *********changed this from 2 to 3
```

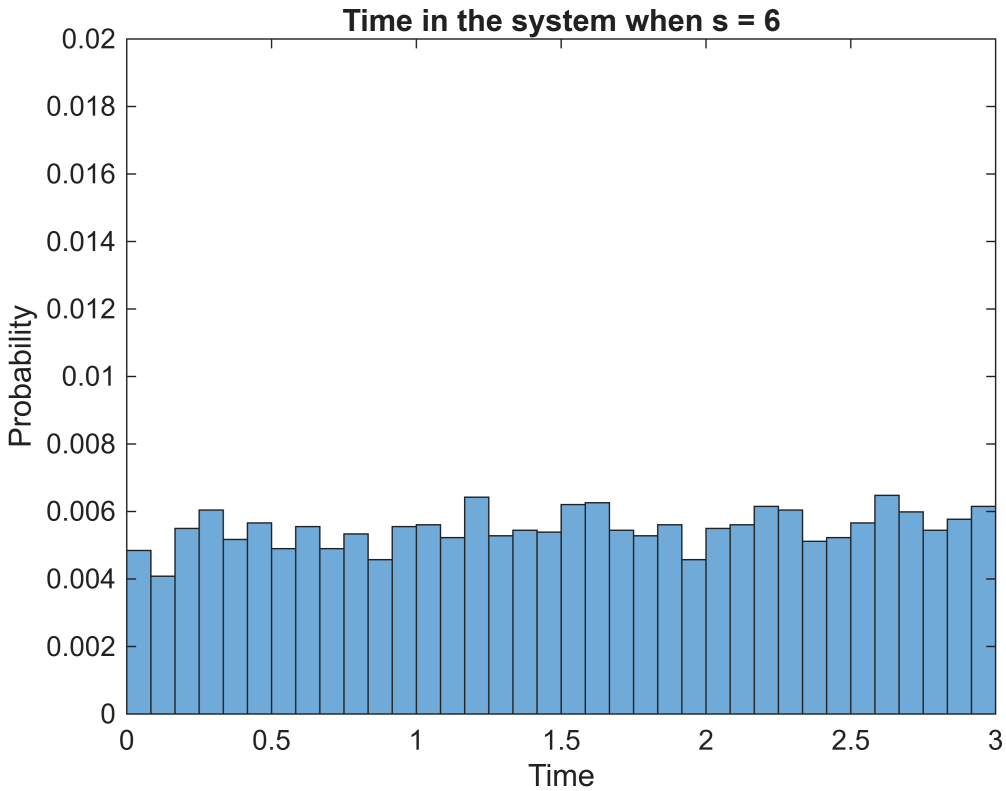
Wait for MATLAB to catch up.

```
pause(2);
```

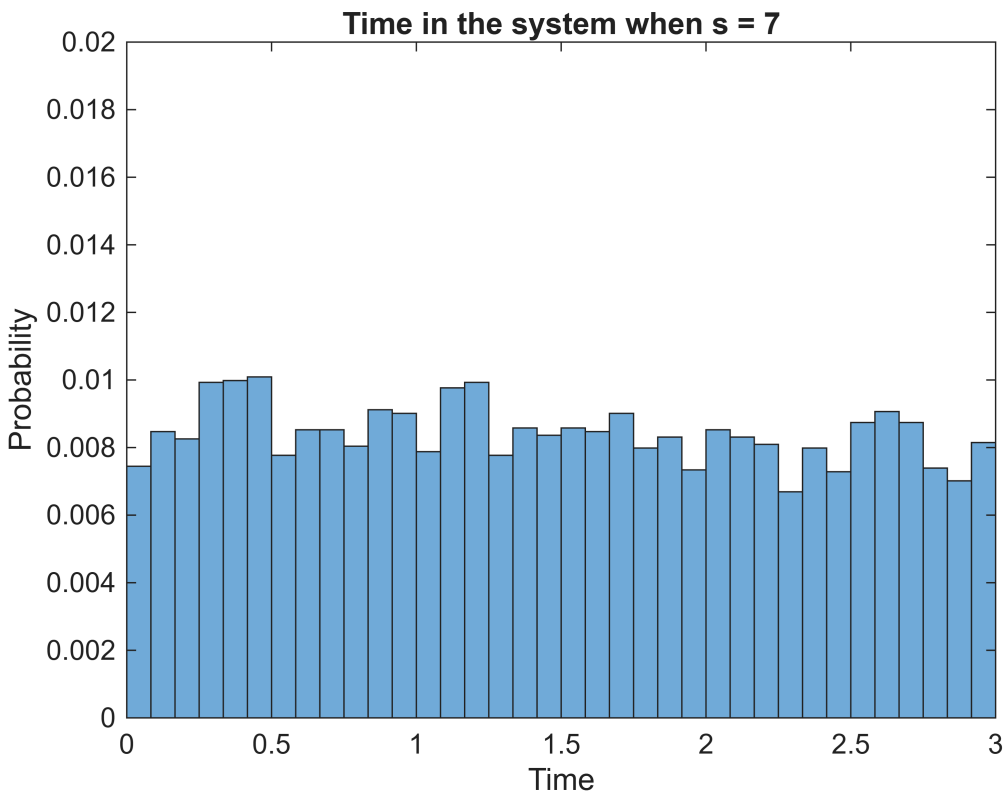
Save the picture.

```
exportgraphics(fig, PictureFolder + filesep + "Time in system
histogram.pdf");
exportgraphics(fig, PictureFolder + filesep + "Time in system
histogram.svg");
```

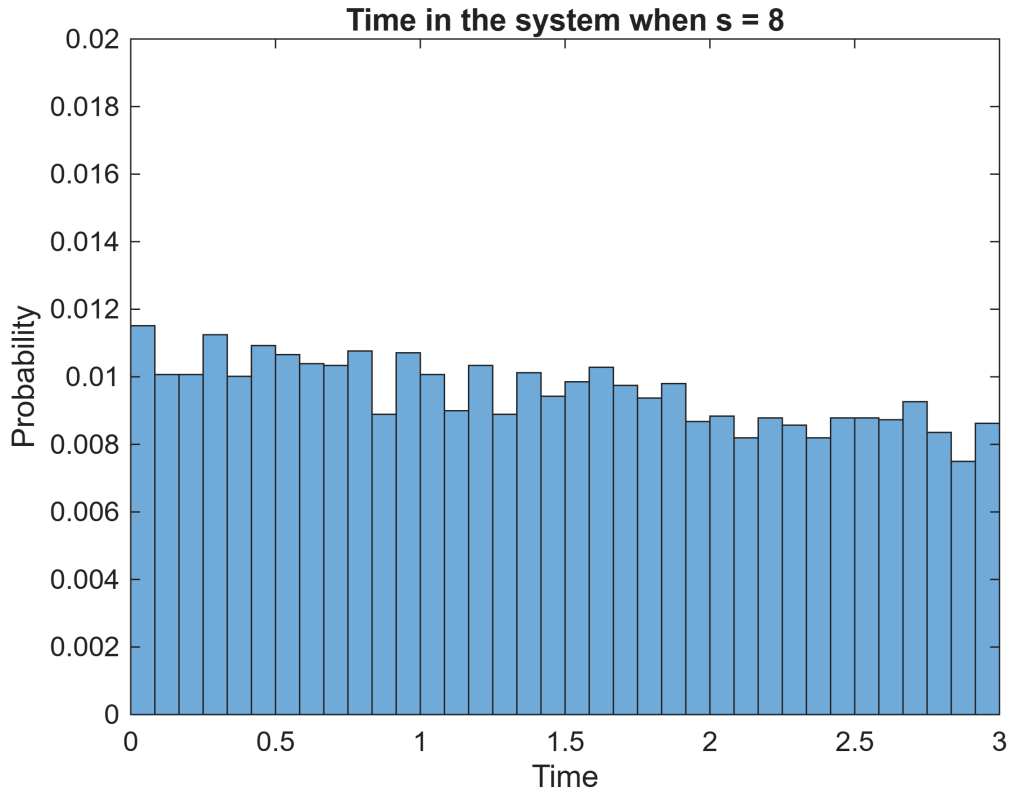
```
Mean time in system: 9.859197
```




```
Mean time in system: 7.309114
```




```
Mean time in system: 6.756690
```




```
disp(" ");
fprintf("Here are the averages of the following values.");
%fprintf("The current number of servers in the simulation calculations is\ns
= %d", s);
fprintf("Number of Customers in system (L): %f\n", mean(NumInSystem));
fprintf("Total time customer spends in system (W): %f\n",
mean(TimeInSystem));
fprintf("Number of customers waiting (L_q): %f\n", mean(NumWaiting));
fprintf("Customer Wait time before service (W_q): %f\n", meanWaitTime);



disp(" ");
disp(" ");
disp(" ");


fprintf("*********************************************************************
*********");
```

```
fprintf("**
       **");
     fprintf("**            Comparing Theoretical and Simulation
Calculations            **");
     fprintf("**                          " + "number of servers k = " + k+
"                         **");

fprintf("**
       **");

fprintf("*********************************************************************
*********");

Error_1 = (abs(Lq - mean(NumWaiting))./Lq)*100;
Error_2 = (abs(L - mean(NumInSystem))./L)*100;
Error_3 = (abs(Wq - mean(ExpectedWaitingTime))./Wq)*100;
Error_4 = (abs(W - mean(TimeInSystem))./W)*100;


% Header with fixed spacing
fprintf('%-18s %-18s %-18s %-18s\n', 'Value Type', 'Theoretical',
'Simulation', 'Error Percent');

% Data rows using fixed-width floats
fprintf('%-18s %-18.4f %-18.4f %-18.4f\n', 'Lq', Lq, mean(NumWaiting),
Error_1);
fprintf('%-18s %-18.4f %-18.4f %-18.4f\n', 'L', L, mean(NumInSystem),
Error_2);
fprintf('%-18s %-18.4f %-18.4f %-18.4f\n', 'Wq', Wq,
mean(ExpectedWaitingTime), Error_3);
fprintf('%-18s %-18.4f %-18.4f %-18.4f\n', 'W', W, mean(TimeInSystem),
Error_4);



%fprintf("Customer Wait time before service (W_q) matrix printed out: ");
%fprintf("(W_q): %f\n ", ExpectedWaitingTime);

longWaits = ExpectedWaitingTime(ExpectedWaitingTime > 5);
disp(" ");

%fprintf("These are the wait times that are greater than 5 minutes:");
%fprintf("(W_q): %f\n ",longWaits);
fprintf("These are the total number of customer wait times:  " +
length(ExpectedWaitingTime));
fprintf("These are the total number of customer wait times W_q > 5 minutes:
" + length(longWaits));
P_of_Wq_greatherthan_5min = length(longWaits)/length(ExpectedWaitingTime);
```

```
disp(" ");
fprintf(2,"The theoretical Wq has us waiting  " + Wq + "  minutes.");
fprintf("When s = " +s+", P( W_q > 5 minutes) = " +
P_of_Wq_greatherthan_5min);
disp(" ");

if s == 6
    fprintf(2,"P(W_q > 5 minutes) > 10 percent ");
elseif s == 7
   fprintf(2,"P(W_q > 5 minutes) < 10 percent");
   fprintf(2, "The minimum number of servers recommended to meet the goal is
s = 7.");
else
   fprintf(2,"P(W_q > 5 minutes) < 10 percent");
end









disp(" ");
disp(" ");
disp(" ");
disp(" ");
disp(" ");



     %P = zeros(1, k-1);    % clears row vector so I can use this again for
the next graph
end % this ends the s = k loop
```

```
Here are the averages of the following values.
Number of Customers in system (L): 7.873257
Total time customer spends in system (W): 9.859197
Number of customers waiting (L_q): 2.898188
Customer Wait time before service (W_q): 3.536028



*****************************************************************************
**                                                                         **
**            Comparing Theoretical and Simulation Calculations            **
**                          number of servers k = 6                        **
**                                                                         **
*****************************************************************************
Value Type         Theoretical        Simulation         Error Percent
Lq                 4.3145             2.8982             32.8261
L                  9.5160             7.8733             17.2631
```

```
Wq                  5.3931             3.5360             34.4338
W                   11.8950            9.8592             17.1149

These are the total number of customer wait times:  18370
These are the total number of customer wait times W_q > 5 minutes:  5053

The theoretical Wq has us waiting  5.3931  minutes.
When s = 6, P( W_q > 5 minutes) = 0.27507

P(W_q > 5 minutes) > 10 percent




Here are the averages of the following values.
Number of Customers in system (L): 5.825295
Total time customer spends in system (W): 7.309114
Number of customers waiting (L_q): 0.826253
Customer Wait time before service (W_q): 1.013057



*****************************************************************************
**                                                                         **
**            Comparing Theoretical and Simulation Calculations            **
**                         number of servers k = 7                         **
**                                                                         **
*****************************************************************************
Value Type          Theoretical        Simulation         Error Percent
Lq                  1.0829             0.8263             23.7034
L                   6.2845             5.8253             7.3071
Wq                  1.3537             1.0131             25.1631
W                   7.8556             7.3091             6.9571

These are the total number of customer wait times:  18532
These are the total number of customer wait times W_q > 5 minutes:  1197

The theoretical Wq has us waiting  1.3537  minutes.
When s = 7, P( W_q > 5 minutes) = 0.064591

P(W_q > 5 minutes) < 10 percent
The minimum number of servers recommended to meet the goal is s = 7.




Here are the averages of the following values.
Number of Customers in system (L): 5.401927
Total time customer spends in system (W): 6.756690
Number of customers waiting (L_q): 0.358740
Customer Wait time before service (W_q): 0.444726



*****************************************************************************
**                                                                         **
**            Comparing Theoretical and Simulation Calculations            **
**                         number of servers k = 8                         **
**                                                                         **
*****************************************************************************
```

```
Value Type         Theoretical        Simulation         Error Percent
Lq                 0.3690             0.3587             2.7933
L                  5.5706             5.4019             3.0281
Wq                 0.4613             0.4447             3.5950
W                  6.9633             6.7567             2.9666

These are the total number of customer wait times:  18674
These are the total number of customer wait times W_q > 5 minutes:  474

The theoretical Wq has us waiting  0.46131  minutes.
When s = 8, P( W_q > 5 minutes) = 0.025383

P(W_q > 5 minutes) < 10 percent
```